

AttritionIQ: AI-Powered Employee Retention Intelligence System

Project Description:

AttritionIQ is an innovative AI-driven HR intelligence system designed to help organizations detect and prevent employee attrition before it happens. The platform includes a Risk Engine module, which produces a composite risk score per employee by blending a trained Random Forest Classifier (60%) with a rule-based behavioral scoring engine (40%), a Pattern Detector, which applies 7 transparent behavioral flags including flight risk, burnout, career stagnation, and compensation gap, and a Retention Advisor, which delivers three specific, personalized AI-generated retention actions tailored to each employee's role, tenure, and active risk signals.

Utilizing Groq's API with the LLaMA 3.3-70B Versatile model, AttritionIQ processes 1,470 IBM HR Analytics employee records to deliver fast, high-quality workforce intelligence. Built with FastAPI and visualized through a polished HTML/CSS/JS dashboard, the platform ensures a seamless and HR-friendly experience. With secure API key management via .env and responsible input handling, AttritionIQ empowers HR teams, managers, and organizations to take proactive, data-driven retention actions with confidence.

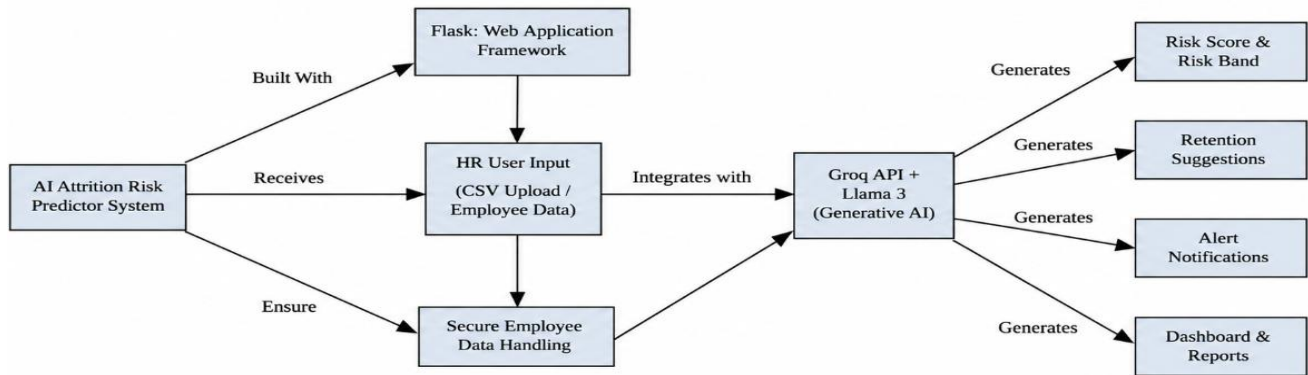
Scenarios

- **Scenario 1:** An HR manager opens the AttritionIQ dashboard. The system has automatically run the full pipeline on startup, loading the IBM HR CSV, engineering features, applying 7 behavioral flags, and computing composite risk scores. The dashboard shows KPI cards (total employees: 1,470, average risk score, high/critical count), a risk distribution doughnut chart, and a table of the top 5 at-risk employees. The HR manager filters by department and immediately identifies that Sales has the highest average composite risk score.
- **Scenario 2:** An HR manager clicks on a HIGH-risk employee — a Sales Executive with a composite score of 78/100. A sliding detail panel opens showing the full profile including 4 years tenure, no promotion in 4+ years, income in the bottom 25th percentile, and high burnout from consistent overtime. The Groq LLaMA 3.3-70B AI has already generated three specific retention actions: schedule a career development conversation, run a compensation benchmarking review, and offer a reduced overtime arrangement for the next 30 days.
- **Scenario 3:** At the end of an analysis run, alert_manager.py scans all employees with composite_score ≥ 65 . For each employee not alerted in the past 24 hours, it dispatches an

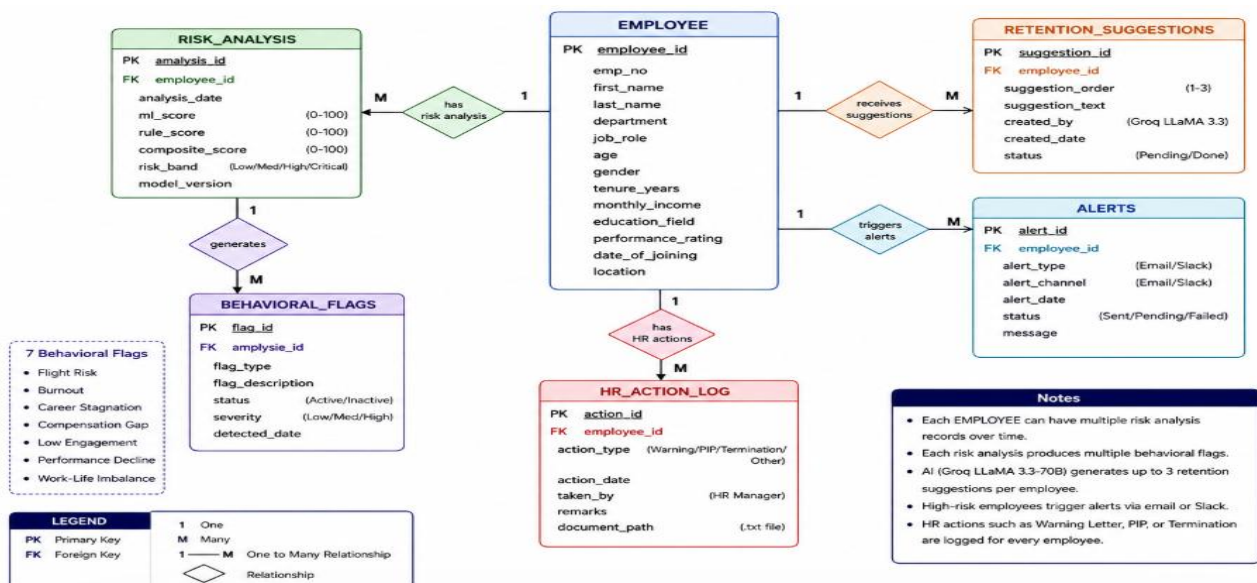
email (via SMTP) and/or a Slack notification containing the employee's role, department, active risk signals, tenure, and AI-generated suggestion. The HR manager can issue a 30-day Performance Improvement Plan for an underperforming CRITICAL-risk employee — auto-generated and downloadable as a .txt file.

- **Scenario 4:** An HR manager uploads a new employee dataset CSV via the Data Source button. The system automatically re-runs the full pipeline — loading, feature engineering, behavioral detection, risk scoring, and AI advisory — on the new data and refreshes all dashboard charts and employee lists without any manual intervention, reflecting the updated workforce risk profile instantly.

Technical Architecture:



ER Diagram:



Pre-requisites:

- **FastAPI Framework Knowledge: FastAPI Documentation**
<https://fastapi.tiangolo.com/>
- **Groq API Familiarity: Groq API Documentation**
<https://console.groq.com/docs>
- **HTML, CSS, and JavaScript Skills: W3Schools HTML/CSS/JavaScript Tutorials**
<https://www.w3schools.com/>
- **Python Programming Proficiency: Python Documentation**
<https://docs.python.org/3/>
- **scikit-learn Machine Learning Library: scikit-learn Documentation**
<https://scikit-learn.org/stable/documentation.html>
- **Pandas Data Analysis Library: Pandas Documentation**
<https://pandas.pydata.org/docs/>
- **Version Control with Git: Git Documentation**
<https://git-scm.com/doc>
- **Development Environment Setup: FastAPI Installation Guide**
<https://fastapi.tiangolo.com/tutorial/>
- **IBM HR Analytics Dataset: Kaggle — IBM HR Analytics Attrition Dataset**
<https://www.kaggle.com/datasets/pavansubhasht/ibm-hr-analytics-attrition-dataset>
- **Machine Learning Fundamentals: Scikit-learn User Guide**
https://scikit-learn.org/stable/user_guide.html

Project Workflow:

Activity 1: Model Selection and Architecture

- **Activity 1.1:** Generate a Groq API key and configure it securely in the project environment.
- **Activity 1.2:** Research and select the appropriate generative AI model from Groq for personalized HR retention advisory.
- **Activity 1.3:** Define the architecture of the application, detailing interactions between the frontend dashboard, FastAPI backend, ML pipeline, and Groq AI integration.
- **Activity 1.4:** Set up the development environment, installing necessary libraries and dependencies for FastAPI, scikit-learn, and the Groq API.

Activity 2: Core Pipeline Development

- **Activity 2.1:** Develop the four core Python modules: `data_loader.py`, `pattern_detector.py`, `risk_engine.py`, and `retention_advisor.py`.
- **Activity 2.2:** Implement the FastAPI backend to manage routing, pipeline orchestration, and user input processing, ensuring smooth API interactions.

Activity 3: App.py Development

- **Activity 3.1:** Write the main application logic in `main.py`, establishing all 10 REST endpoints and integrating the full ML pipeline and AI advisory system.

Activity 4: Frontend Development

- **Activity 4.1:** Design and develop the HR analytics dashboard using HTML, CSS, and Vanilla JS, ensuring a responsive and intuitive layout for non-technical HR professionals.
- **Activity 4.2:** Create dynamic dashboard views — Overview, Employees, Alerts, and Analytics tabs — that consume FastAPI endpoints and render results in real time.

Activity 5: Deployment

- **Activity 5.1:** Prepare the application for local deployment by configuring the server environment and ensuring all dependencies are correctly installed.
- **Activity 5.2:** Deploy the application locally using `uvicorn` and verify all 10 REST endpoints, pipeline accuracy, and dashboard rendering across browsers.

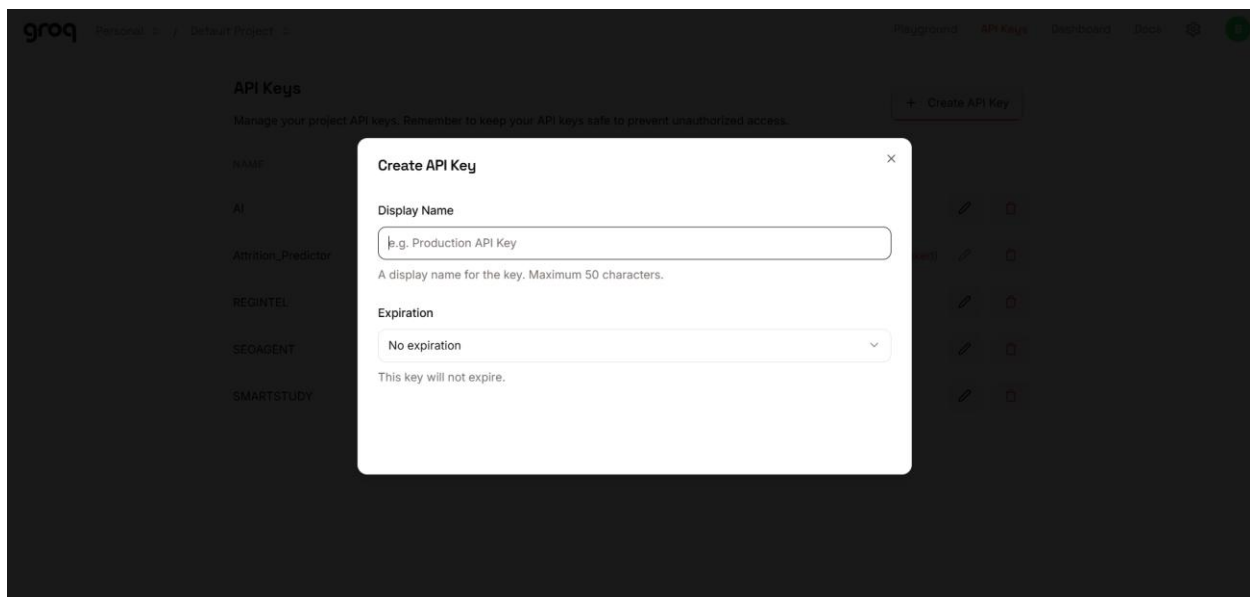
Milestone 1: Model Selection and Architecture

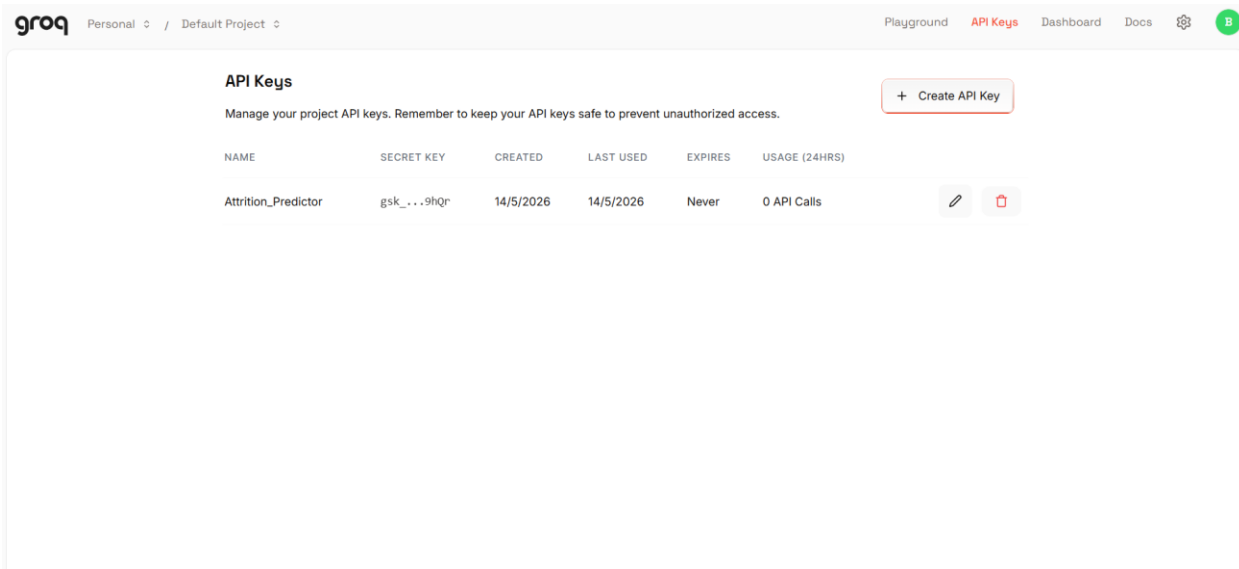
In this milestone, we focus on selecting the appropriate generative AI model from Groq for our HR retention advisory needs. This involves researching the capabilities and performance of various models that Groq offers, ensuring that the chosen model aligns well with the application's objectives of generating personalized, role-specific retention recommendations for high-risk employees.

Activity 1.1: Generate a Groq API Key

Before the application can generate AI-powered retention suggestions, you need a valid Groq API key. Follow the steps below to create one and connect it securely to the project.

- **Step 1 — Create a Groq Account:** Visit <https://console.groq.com> and sign up for a free account using your Google account or email address.
- **Step 2 — Navigate to API Keys:** After logging in, click on your profile icon in the top-right corner and select "API Keys" from the dropdown menu, or go directly to the API Keys page.
- **Step 3 — Create a New API Key:** Click the "Create API Key" button. Give your key a descriptive name (e.g., attritioniq-key) so it is easy to identify later.
- **Step 4 — Copy the API Key:** Give a "Display Name" and optionally set an expiration, then click "Submit". Copy the key immediately — you will not be able to view it again after closing the dialog.
- **Step 5 — Add the Key to Your Project:** Create a .env file in the root of your project directory and paste the key as shown below:





- **Step 6 — Verify the Key is Loaded:** The main.py file loads this key automatically at startup using python-dotenv:
- **Important — Never Commit Your API Key:** Add .env to your .gitignore file to prevent accidentally pushing the key to a public repository:

Activity 1.2: Research and Select the Appropriate Generative AI Model

- **Understand the Project Requirements:** Review the specific needs of the AttritionIQ application, focusing on the types of content it will generate (personalized retention recommendations tailored to role, department, tenure, and active behavioral flags).
- **Explore Groq's Model Documentation:** Visit the Groq documentation to examine the available LLM models, including their functionalities, context windows, speed, and limitations.
- **Evaluate Model Performance:** Compare models based on response quality, generation speed, and relevance to HR advisory and professional recommendations.
- **Select the Optimal Model:** Choose llama-3.3-70b-versatile for its balance of professional output quality and generation speed, set with a temperature of 0.3 for consistent, professional retention advice and max_tokens of 500.

Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Create a visual representation of the application architecture, including components like the frontend dashboard (HTML, CSS, Vanilla JS), FastAPI backend, ML pipeline (scikit-learn Random Forest), and Groq API integration.
- **Detail Frontend Functionality:** Outline how HR users interact with the application — viewing the risk dashboard, drilling into employee profiles, filtering by department and risk band, and triggering HR actions (Warning Letter, PIP, Termination).

- **Outline Backend Responsibilities:** Specify how the FastAPI backend handles incoming requests to all 10 REST endpoints, orchestrates the full pipeline via `_run_pipeline()`, and bridges the frontend, ML pipeline, and Groq AI.
- **Describe AI Integration Points:** Define how `retention_advisor.py` constructs dynamic prompts from employee role, department, tenure, composite score, and active behavioral flags, calls the Groq API concurrently via `ThreadPoolExecutor`, and caches responses in `suggestions_cache.json`.

Activity 1.4: Set Up the Development Environment

- **Install Python and Pip:** Ensure Python 3.10+ is installed on your machine along with pip for dependency management.
- **Create a Virtual Environment:** Set up a virtual environment using `venv` to isolate project dependencies:

```
C:\WINDOWS\system32\cmd. x + v
(attrition_env) C:\Users\baiya>cd C:\Users\baiya\OneDrive\Documents\Projects\attrition_risk_predictor
(attrition_env) C:\Users\baiya\OneDrive\Documents\Projects\attrition_risk_predictor>pip install -r requirements.txt
Requirement already satisfied: fastapi==0.115.5 in c:\users\baiya\anaconda3\envs\attrition_env\lib\site-packages (from -r requirements.txt (line 1)) (0.115.5)
Requirement already satisfied: uvicorn==0.32.1 in c:\users\baiya\anaconda3\envs\attrition_env\lib\site-packages (from uvicorn[standard]==0.32.1->-r requirements.txt (line 2)) (0.32.1)
Requirement already satisfied: pandas==2.2.3 in c:\users\baiya\anaconda3\envs\attrition_env\lib\site-packages (from -r requirements.txt (line 3)) (2.2.3)
Requirement already satisfied: numpy==1.26.4 in c:\users\baiya\anaconda3\envs\attrition_env\lib\site-packages (from -r requirements.txt (line 4)) (1.26.4)
Requirement already satisfied: scikit-learn==1.5.2 in c:\users\baiya\anaconda3\envs\attrition_env\lib\site-packages (from -r requirements.txt (line 5)) (1.5.2)
Requirement already satisfied: joblib==1.4.2 in c:\users\baiya\anaconda3\envs\attrition_env\lib\site-packages (from -r requirements.txt (line 6)) (1.4.2)
Collecting groq==0.11.0 (from -r requirements.txt (line 7))
  Using cached groq-0.11.0-py3-none-any.whl.metadata (13 kB)
Requirement already satisfied: requests==2.32.3 in c:\users\baiya\anaconda3\envs\attrition_env\lib\site-packages (from -r requirements.txt (line 8)) (2.32.3)
Requirement already satisfied: starlette<0.42.0,>=0.40.0 in c:\users\baiya\anaconda3\envs\attrition_env\lib\site-packages (from fastapi==0.115.5->-r requirements.txt (line 1)) (0.41.3)
Requirement already satisfied: pydantic!=1.8,!=1.8.1,!=2.0.0,!=2.0.1,!=2.1.0,<3.0.0,>=1.7.4 in c:\users\baiya\anaconda3\envs\attrition_env\lib\site-packages (from fastapi==0.115.5->-r requirements.txt (line 1)) (2.13.4)
Requirement already satisfied: typing-extensions>=4.8.0 in c:\users\baiya\appdata\roaming\python\python310\site-packages (from fastapi==0.115.5->-r requirements.txt (line 1)) (4.15.0)
Requirement already satisfied: click>=7.0 in c:\users\baiya\anaconda3\envs\attrition_env\lib\site-packages (from uvicorn==0.32.1->uvicorn[standard]==0.32.1->-r requirements.txt (line 2)) (8.3.3)
Requirement already satisfied: h11>=0.8 in c:\users\baiya\anaconda3\envs\attrition_env\lib\site-packages (from uvicorn==0.32.1->uvicorn[standard]==0.32.1->-r requirements.txt (line 2)) (0.16.0)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\baiya\anaconda3\envs\attrition_env\lib\site-packages (from pandas==2.2.3->-r requirements.txt (line 3)) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\baiya\anaconda3\envs\attrition_env\lib\site-packages (from pandas==2.2.3->-r requirements.txt (line 3)) (2026.2)
Requirement already satisfied: tzdata>=2022.7 in c:\users\baiya\anaconda3\envs\attrition_env\lib\site-packages (from pandas==2.2.3->-r requirements.txt (line 3)) (2026.2)
Requirement already satisfied: scipy>=1.6.0 in c:\users\baiya\anaconda3\envs\attrition_env\lib\site-packages (from scikit-learn==1.5.2->-r requirements.txt (line 5)) (1.15.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in c:\users\baiya\anaconda3\envs\attrition_env\lib\site-packages (from scikit-learn==1.5.2->-r requirements.txt (line 5)) (3.6.0)
Requirement already satisfied: anyio<5,>=3.5.0 in c:\users\baiya\anaconda3\envs\attrition_env\lib\site-packages (from groq==0.11.0->-r requirements.txt (line 7)) (4.13.0)
Requirement already satisfied: distro<2,>=1.7.0 in c:\users\baiya\anaconda3\envs\attrition_env\lib\site-packages (from groq==0.11.0->-r requirements.txt (line 7)) (1.9.0)
Requirement already satisfied: httpx<1,>=0.23.0 in c:\users\baiya\anaconda3\envs\attrition_env\lib\site-packages (from groq==0.11.0->-r requirements.txt (line 7)) (0.23.0)
```

- **Install FastAPI and Uvicorn:** Use pip to install the async web framework and ASGI server:
- **Install scikit-learn and Data Libraries:** Install the ML and data processing dependencies:
- **Install Groq SDK:** Install the Groq Python library to interact with the Groq API:
- **Install python-dotenv:** Install dotenv support for secure API key management:
- **Configure the API Key:** Create a `.env` file in the project root and add your Groq API key:

```
1  ✓ from groq import Groq
2    import os
3    from dotenv import load_dotenv
4
5    load_dotenv()
6
7    client = Groq(api_key=os.getenv("GROQ_API_KEY"))
8  ✓ resp = client.chat.completions.create(
9      |     model='llama-3.3-70b-versatile',
10     |     max_tokens=10,
11     |     messages=[{'role': 'user', 'content': 'ping'}]
12 )
13 print("Groq API connected successfully!")
14 print("Response:", resp.choices[0].message.content)
```

- **Set Up Application Structure:** Create the project directory structure including the main application file (main.py), core pipeline modules (data_loader.py, pattern_detector.py, risk_engine.py, retention_advisor.py, alert_manager.py, config.py), the IBM HR dataset (employees.csv), and the frontend dashboard (index.html).

Milestone 2: Core Pipeline Development

Activity 2.1: Develop Core Analytics and Generation Features

Implement the four core pipeline modules — data loading, behavioral detection, risk scoring, and AI advisory — as a unified, sequentially orchestrated system.

- **Data Loader (data_loader.py):** Loads the IBM HR Analytics CSV, validates schema against 17 required columns, deduplicates records, and engineers four composite features: engagement_score (mean of JobSatisfaction, EnvironmentSatisfaction, WorkLifeBalance, JobInvolvement), stagnation_ratio (YearsSinceLastPromotion / YearsAtCompany), income_percentile (percentile rank within same JobRole), and burnout_score (OverTime_Binary + (4 - WorkLifeBalance)).

```
def load_and_preprocess(file_path: Optional[str] = None) -> pd.DataFrame:
    # Use provided path, else default from config
    source = file_path if file_path else str(DATA_PATH)

    try:
        df = pd.read_csv(source)
    except Exception as e:
        raise FileNotFoundError(f"Could not load data from '{source}': {e}")

    missing = [c for c in REQUIRED_COLUMNS if c not in df.columns]
    if missing:
        raise ValueError(
            f"Missing columns: {missing}. "
            f"Please upload a valid IBM HR Analytics dataset with all required columns."
        )

    df = df.drop_duplicates(subset="EmployeeNumber").copy()

    df["Attrition_Binary"] = (df["Attrition"] == "Yes").astype(int)
    df["OverTime_Binary"] = (df["OverTime"] == "Yes").astype(int)

    df[NUMERIC_COLS] = df[NUMERIC_COLS].apply(pd.to_numeric, errors="coerce")
    df[NUMERIC_COLS] = df[NUMERIC_COLS].fillna(df[NUMERIC_COLS].median())
```

```
df[NUMERIC_COLS] = df[NUMERIC_COLS].apply(pd.to_numeric, errors="coerce")
df[NUMERIC_COLS] = df[NUMERIC_COLS].fillna(df[NUMERIC_COLS].median())

# Derived features
df["engagement_score"] = df[["JobSatisfaction", "EnvironmentSatisfaction",
                             "WorkLifeBalance", "JobInvolvement"]].mean(axis=1)

df["stagnation_ratio"] = df["YearsSinceLastPromotion"] / df["YearsAtCompany"].replace(0, 1)
df["income_percentile"] = df.groupby("JobRole")["MonthlyIncome"].rank(pct=True)
df["burnout_score"] = df["OverTime_Binary"] + (4 - df["WorkLifeBalance"])

logger.info(f"Loaded {len(df)} employees from '{source}'")
return df

def get_feature_matrix(df: pd.DataFrame) -> pd.DataFrame:
    return df[FEATURE_COLS]
```

- **Pattern Detector (pattern_detector.py):** Scans every employee row and applies 7 boolean behavioral flags based on threshold logic: absenteeism_flag, low_engagement_flag, career_stagnation_flag, flight_risk_flag, compensation_flag, high_burnout_flag, and performance_drop_flag. Flags are transparent and immediately interpretable by HR managers.

```
FLAG_DESCRIPTIONS = {
    "absenteeism_flag": "High absenteeism / burnout-driven absence",
    "low_engagement_flag": "Low overall engagement",
    "performance_drop_flag": "Below-average performance rating",
    "career_stagnation_flag": "No promotion in 4+ years",
    "flight_risk_flag": "Has worked at 4+ companies",
    "compensation_flag": "Paid below 25th percentile for role",
    "high_burnout_flag": "Overtime + poor work-life balance",
}

def detect_patterns(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()

    df["absenteeism_flag"] = (df["OverTime_Binary"] == 1) & (df["WorkLifeBalance"] <= 2)
    df["low_engagement_flag"] = df["engagement_score"] <= 2.0
    df["performance_drop_flag"] = df["PerformanceRating"] <= 2
    df["career_stagnation_flag"] = df["YearsSinceLastPromotion"] >= 4
    df["flight_risk_flag"] = df["NumCompaniesWorked"] >= 4
    df["compensation_flag"] = df["income_percentile"] <= 0.25
    df["high_burnout_flag"] = df["burnout_score"] >= 3

    flag_cols = [c for c in df.columns if c.endswith("_flag")]
    df["active_flag_count"] = df[flag_cols].sum(axis=1)

    logger.info(f"Avg flags/employee: {df['active_flag_count'].mean():.2f}")
    return df
```

- **Risk Engine (risk_engine.py):** The analytical core. Blends rule-based score (40% weight) with Random Forest ML probability (60% weight) to produce a composite 0-100 risk score. Trains a 200-tree Random Forest with class_weight='balanced' and classifies into four bands: LOW (0-39), MEDIUM (40-64), HIGH (65-79), and CRITICAL (80-100).

```
def compute_risk_scores(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    df["rule_score"] = _rule_score(df)
    df["ml_score"] = _ml_score(df) * 100
    df["composite_score"] = (df["rule_score"] * RULE_WEIGHT + df["ml_score"] * ML_WEIGHT).round(1)
    df["risk_band"] = df["composite_score"].apply(_band)

    logger.info(f"Risk bands: {df['risk_band'].value_counts().to_dict()}")
    return df

def train_model(df: pd.DataFrame) -> RandomForestClassifier:
    X, y = get_feature_matrix(df, df["Attrition_Binary"])
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                         random_state=42, stratify=y)
    model = RandomForestClassifier(
        n_estimators=200,
        max_depth=10,
        min_samples_leaf=5,
        class_weight="balanced", # IBM dataset is imbalanced (~16% attrition)
        random_state=42,
        n_jobs=-1,
    )
    model.fit(X_train, y_train)

    logger.info(f"\n{classification_report(y_test, model.predict(X_test), target_names=['Stayed', 'Left'])}")
    joblib.dump(model, MODEL_PATH)
    logger.info(f"Model saved -> {MODEL_PATH}")
    return model
```

```
def _rule_score(df: pd.DataFrame) -> pd.Series:
    s = pd.Series(0.0, index=df.index)
    s += df["OverTime_Binary"] * RULE_WEIGHTS["overtime"]
    s += (df["JobSatisfaction"] <= 2).astype(int) * RULE_WEIGHTS["low_job_satisfaction"]
    s += (df["YearsSinceLastPromotion"] >= 4).astype(int) * RULE_WEIGHTS["no_recent_promotion"]
    s += (df["WorkLifeBalance"] <= 2).astype(int) * RULE_WEIGHTS["poor_work_life_balance"]
    s += (df["EnvironmentSatisfaction"] <= 2).astype(int) * RULE_WEIGHTS["poor_env_satisfaction"]
    s += (df["NumCompaniesWorked"] >= 4).astype(int) * RULE_WEIGHTS["high_company_count"]
    return s.clip(0, 100)

def _ml_score(df: pd.DataFrame) -> pd.Series:
    model = joblib.load(MODEL_PATH) if MODEL_PATH.exists() else train_model(df)
    probs = model.predict_proba(get_feature_matrix(df))[:, 1]
    return pd.Series(probs, index=df.index)

def _band(score: float) -> str:
    for band, (lo, hi) in RISK_BANDS.items():
        if lo <= score <= hi:
            return band
    return "CRITICAL"
```

- **Retention Advisor (retention_advisor.py):** For every employee scoring ≥ 70 , calls Groq LLaMA 3.3-70B concurrently via ThreadPoolExecutor (5 workers). Caches all responses in suggestions_cache.json and falls back to flag-keyed pre-written suggestions on API failure.

```
def generate_suggestions_for_all(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    df["retention_suggestion"] = ""

    at_risk = df[df["composite_score"] >= SUGGESTION_THRESHOLD]
    cache = _load_cache()
    to_call = []

    for idx, row in at_risk.iterrows():
        cache_key = f"{row['EmployeeNumber']}_{row['composite_score']:.1f}"
        if cache_key in cache:
            df.at[idx, "retention_suggestion"] = cache[cache_key]
        else:
            to_call.append((idx, row, cache_key))
```

Activity 2.2: Implement the FastAPI Backend

Define Routes in FastAPI:

- Set up all 10 routes in main.py for the dashboard home page, pipeline analysis, employee lookup, alert management, and HR action document generation.

Process User Input:

- Use FastAPI's request parsing and Pydantic models (TerminateRequest, WarningRequest, PipRequest) to validate and process all incoming JSON data.
- Validate that employee IDs exist in the processed dataset before executing any HR action endpoints.

Integrate Pipeline and API Calls:

- Within the /analyze route handler, call `_run_pipeline()` to orchestrate the full sequence: `load_and_preprocess()` → `detect_patterns()` → `compute_risk_scores()` → `generate_suggestions_for_all()` → `dispatch_alerts()`.
- Submit retention advisory prompts to the Groq API using `client.chat.completions.create()` with the llama-3.3-70b-versatile model at temperature=0.3.
- Parse and validate structured prompt responses, ensuring all required retention actions are present before caching and returning to the frontend.

```
PS C:\Users\baiya\OneDrive\Documents\Projects\attrition_risk_predictor> uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\baiya\\OneDrive\\Documents\\Projects\\attrition_risk_predictor']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [30992] using WatchFiles
INFO: Started server process [10144]
INFO: Waiting for application startup.
2026-06-25 11:32:13,266 | main | INFO | Running pipeline on startup...
```

Milestone 3: main.py Development

Milestone 3 focuses on creating and refining the core logic of the main.py file, which serves as the backbone of the AttritionIQ application. In this milestone, you'll set up each feature's functionality through FastAPI routes, establish reliable pipeline orchestration and API interaction, and conduct unit testing to ensure each component performs as expected.

Activity 3.1: Writing the Main Application Logic in main.py

Define the Core Routes in main.py:

- Establish routes for AttritionIQ's 10 endpoints, each serving a distinct purpose:
 - / — Home page route that serves the main index.html dashboard
 - /analyze (POST) — triggers full pipeline re-run and returns risk distribution summary
 - /employees (GET) — paginated, filterable employee list sorted by composite_score descending
 - /employees/{id} (GET) — full employee detail with all flags and AI suggestion
 - /alerts (GET) — all employees above ALERT_THRESHOLD=65, sorted by risk
 - /dashboard-summary (GET) — aggregated stats including age/tenure group risk and flag prevalence
 - /upload-datasource (POST) — accepts CSV file upload and re-runs full pipeline on new data
 - /employees/{id}/terminate (POST) — logs termination with reason and authorization
 - /employees/{id}/warning (POST) — auto-generates formal warning letter as .txt download
 - /employees/{id}/pip (POST) — auto-generates 30-day Performance Improvement Plan as .txt download

Lifespan and Startup Logic:

```
# — Lifespan —————
@asynccontextmanager
async def lifespan(app: FastAPI):
    logger.info("Running pipeline on startup...")
    _state["df"] = _run_pipeline(send_alerts=False)
    yield

app = FastAPI(title="Attrition Risk Predictor", version="1.0.0", lifespan=lifespan)
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"], allow_headers=["*"])
```

Set Up Route Handlers for Each Feature:

- For the /analyze route, implement logic that triggers _run_pipeline(send_alerts=True) and returns a risk distribution summary.
- For /employees, implement pagination and filtering by risk_band and department, sorted by composite_score descending.

- Route `fetch()` calls from the dashboard frontend to the respective FastAPI endpoints, which process data and return structured JSON responses.

Define Prompt Engineering Helpers:

- `TONE_DESCRIPTIONS` equivalent — each employee's prompt is dynamically constructed with role, department, tenure, composite score, and active flag descriptions as contextual elements.

```
FLAG_DESCRIPTIONS = {
    "absenteeism_flag": "High absenteeism / burnout-driven absence",
    "low_engagement_flag": "Low overall engagement",
    "performance_drop_flag": "Below-average performance rating",
    "career_stagnation_flag": "No promotion in 4+ years",
    "flight_risk_flag": "Has worked at 4+ companies",
    "compensation_flag": "Paid below 25th percentile for role",
    "high_burnout_flag": "Overtime + poor work-life balance",
}

def detect_patterns(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()

    df["absenteeism_flag"] = (df["OverTime_Binary"] == 1) & (df["WorkLifeBalance"] <= 2)
    df["low_engagement_flag"] = df["engagement_score"] <= 2.0
    df["performance_drop_flag"] = df["PerformanceRating"] <= 2
    df["career_stagnation_flag"] = df["YearsSinceLastPromotion"] >= 4
    df["flight_risk_flag"] = df["NumCompaniesWorked"] >= 4
    df["compensation_flag"] = df["income_percentile"] <= 0.25
    df["high_burnout_flag"] = df["burnout_score"] >= 3

    flag_cols = [c for c in df.columns if c.endswith("_flag")]
    df["active_flag_count"] = df[flag_cols].sum(axis=1)

    logger.info(f"Avg flags/employee: {df['active_flag_count'].mean():.2f}")
    return df

def get_active_flags(row: pd.Series) -> list[str]:
    return [desc for flag, desc in FLAG_DESCRIPTIONS.items() if row.get(flag, False)]
```

Integrate the Groq AI Responses:

- Call `client.chat.completions.create()` with a system message instructing the model to act as a senior HR consultant and return exactly 3 numbered, single-line actions HR should take this week.
- Temperature set to 0.3 for consistent, professional outputs. Responses cached in `suggestions_cache.json` using `EmployeeNumber + composite_score` as a composite key.
- On API failure, a rule-based fallback map provides pre-written suggestions keyed by flag type, ensuring the system always returns actionable recommendations.

- HR Action endpoints (/terminate, /warning, /pip) log all actions to separate JSON audit files (terminations_log.json, warnings_log.json, pip_log.json) and generate downloadable formal HR documents.

Milestone 4: Frontend Dashboard Development

Milestone 4 focuses on developing a professional and data-rich HR analytics interface for AttritionIQ. This involves designing a dark-themed dashboard with responsive HTML and CSS to enhance usability, and creating dynamic chart rendering with Chart.js and Vanilla JavaScript to display risk distributions, department analytics, and employee profiles based on live API data.

Activity 4.1: Designing and Developing the Dashboard Interface

Set Up the Base HTML Structure:

- Develop the main HTML file (index.html) as the single-page dashboard interface, including a sidebar navigation with AttritionIQ brand logo, and four primary tabs: Dashboard, Employees, Alerts, and Analytics.
- Use semantic HTML elements to keep the structure organized and ensure the interface is accessible for non-technical HR professionals.
- Integrate Font Awesome icons for navigation and action buttons, and Plus Jakarta Sans / Inter from Google Fonts for clean professional typography.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>HR Analytics | Attrition Intelligence Dashboard</title>
  <link href="https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@400;500;600;700;800&family=Inter:wght@400;500;600&display=swap" rel="stylesheet">
  <link href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css" rel="stylesheet">
  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
  <style>
    :root {
      --bg-color: #f8f9fc;
      --sidebar-bg: #0f172a;
      --primary: #4f46e5;
      --primary-hover: #4338ca;
      --text-main: #0f172a;
      --text-muted: #64748b;

      --critical-high: #ef4444;
      --medium: #f59e0b;
      --low: #10b981;

      --male: #3b82f6;
      --female: #ec4899;

      --card-radius: 16px;
      --card-shadow: 0 4px 24px -4px rgba(15, 23, 42, 0.04), 0 2px 8px -2px rgba(15, 23, 42, 0.02);
      --hover-shadow: 0 12px 32px -4px rgba(15, 23, 42, 0.08), 0 4px 12px -2px rgba(15, 23, 42, 0.04);
      --card-border: 1px solid #0f172a;
      --transition: all 250ms cubic-bezier(0.4, 0, 0.2, 1);
    }
  </style>
</head>
```

Design a Responsive Layout Using CSS:

- Create a CSS stylesheet implementing a dark indigo/slate theme using CSS Custom Properties for consistent theming, shadows, and transitions across all components.
- Use flexbox and grid layouts to organize the Overview tab into a 6-KPI-card row, risk distribution charts, and a top-5 at-risk employee table.
- Add responsive breakpoints to ensure the layout adapts across desktop, tablet, and mobile screen sizes.

Create Separate UI Components for Each Dashboard Section:

- KPI Cards: six metric cards showing total employees, active employees, total risk count, attrition rate, average risk score, and critical alerts — each with an animated icon.
- Risk Charts: a doughnut chart for risk distribution by department and gender, a bar chart for department risk levels (LOW/MEDIUM/HIGH/CRITICAL stacked), and a line chart for risk trend by years at company.
- Employee Cards: searchable, filterable rows with risk band badges, flag count chips, and inline Warning Letter, PIP, and Terminate action buttons on the Alerts tab.

Activity 4.2: Creating Dynamic Data Rendering with JavaScript

Handle API Data Fetching Asynchronously:

- Attach `fetch()` calls to each dashboard tab that retrieve live data from the FastAPI backend endpoints on load and on the Re-run Analysis button click.
- During loading, disable the Re-run Analysis button and show an animated loading state in place of the button text.

Render Results Dynamically:

- Implement dynamic DOM injection to render KPI values, Chart.js visualizations, employee rows, alert cards, and employee detail side-panels from API response data.
- After rendering, update the Last Refreshed timestamp in the data pipeline status bar at the bottom of the dashboard.

Restore UI State After Generation:

- In the finally block of each async handler, re-enable action buttons and restore the original button text and icon regardless of API success or failure.
- Toast notifications (3-second auto-dismiss) confirm all async operations including pipeline re-runs, HR document generation, and data source uploads.

Milestone 5: Deployment

In Milestone 5, the focus is on deploying the AttritionIQ application first locally to verify correct pipeline operation and dashboard rendering, and then verifying all 10 REST endpoints through FastAPI's auto-generated Swagger UI. This involves configuring the server environment, managing dependencies, and launching the full stack for real-world HR use.

Activity 5.1: Preparing the Application for Local Deployment

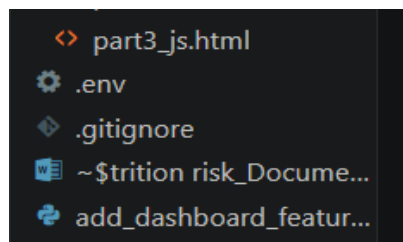
Set Up a Virtual Environment:

- Create and activate a virtual environment, then install all required dependencies from requirements.txt:

```
python -m venv env
source env/bin/activate # For macOS/Linux
env\Scripts\activate    # For Windows
pip install -r requirements.txt
```

Configure Environment Variables:

- Create a .env file in the project root and add your Groq API key. The app loads this automatically using python-dotenv at startup:



```
GROQ_API_KEY=your_groq_api_key_here
EMAIL_ENABLED=False
SLACK_ENABLED=False
```

Activity 5.2: Local Testing and Verification

Start the FastAPI Server:

- Run the application locally using:

```
uvicorn main:app --reload --port 8000
```

- Once running, open a browser and navigate to <http://127.0.0.1:8000> to access the dashboard, and <http://127.0.0.1:8000/docs> for the auto-generated Swagger UI.
- Interact with the dashboard — verify the full pipeline runs on startup, all 6 KPI cards populate, risk distribution charts render correctly, and the top-5 at-risk employees table loads.
- Test all 10 API endpoints using FastAPI's Swagger UI at /docs to verify correct request/response formats and pipeline data flow.

```
PS C:\Users\baiya\OneDrive\Documents\Projects\attrition_risk_predictor> uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\baiya\\OneDrive\\Documents\\Projects\\attrition_risk_predictor']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [30992] using WatchFiles
INFO: Started server process [10144]
INFO: Waiting for application startup.
2026-06-25 11:32:13,266 | main | INFO | Running pipeline on startup...
```

Activity 5.3: Endpoint and Pipeline Validation

Validate the Full Pipeline:

- Confirm the pipeline runs on startup, loading all 1,470 employee records, applying 7 behavioral flags, computing composite scores, and generating AI suggestions for HIGH/CRITICAL employees.
- Verify startup logs confirm: data loaded, avg flags/employee computed, risk band distribution shown (e.g., LOW: 1089, MEDIUM: 303, HIGH: 53, CRITICAL: 25), and AI suggestions generated/loaded from cache.

```
C:\WINDOWS\system32\cmd. X + v
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

(attrition_env) C:\Users\baiya>C:/Users/baiya/OneDrive/Documents/Projects/attrition_risk_predictor/index.html

(attrition_env) C:\Users\baiya>cd C:/Users/baiya/OneDrive/Documents/Projects/attrition_risk_predictor

(attrition_env) C:\Users\baiya\OneDrive\Documents\Projects\attrition_risk_predictor>uvicorn main:app --reload --port 8000
INFO: Will watch for changes in these directories: ['C:\\Users\\baiya\\OneDrive\\Documents\\Projects\\attrition_risk_predictor']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [36552] using WatchFiles
INFO: Started server process [36888]
INFO: Waiting for application startup.
2026-05-17 15:37:40,700 | main | INFO | Running pipeline on startup...
2026-05-17 15:37:40,836 | data_loader | INFO | Loaded 1470 employees from 'C:\\Users\\baiya\\OneDrive\\Documents\\Projects\\attrition_risk_predictor\\employees.csv'
2026-05-17 15:37:40,872 | pattern_detector | INFO | Avg fflags/employee: 1.07
2026-05-17 15:37:41,185 | risk_engine | INFO | Risk bands: {'LOW': 1089, 'MEDIUM': 303, 'HIGH': 53, 'CRITICAL': 25}
2026-05-17 15:37:41,203 | retention_advisor | INFO | Suggestions: 34 from cache, 0 calling Groq
INFO: Application startup complete.
```

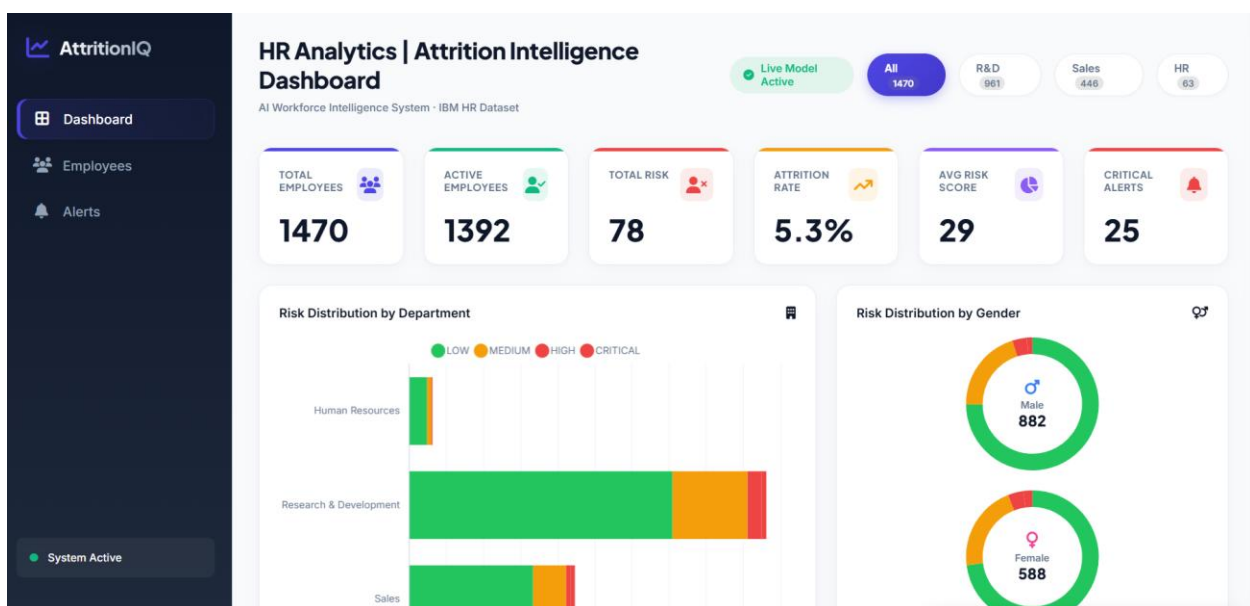
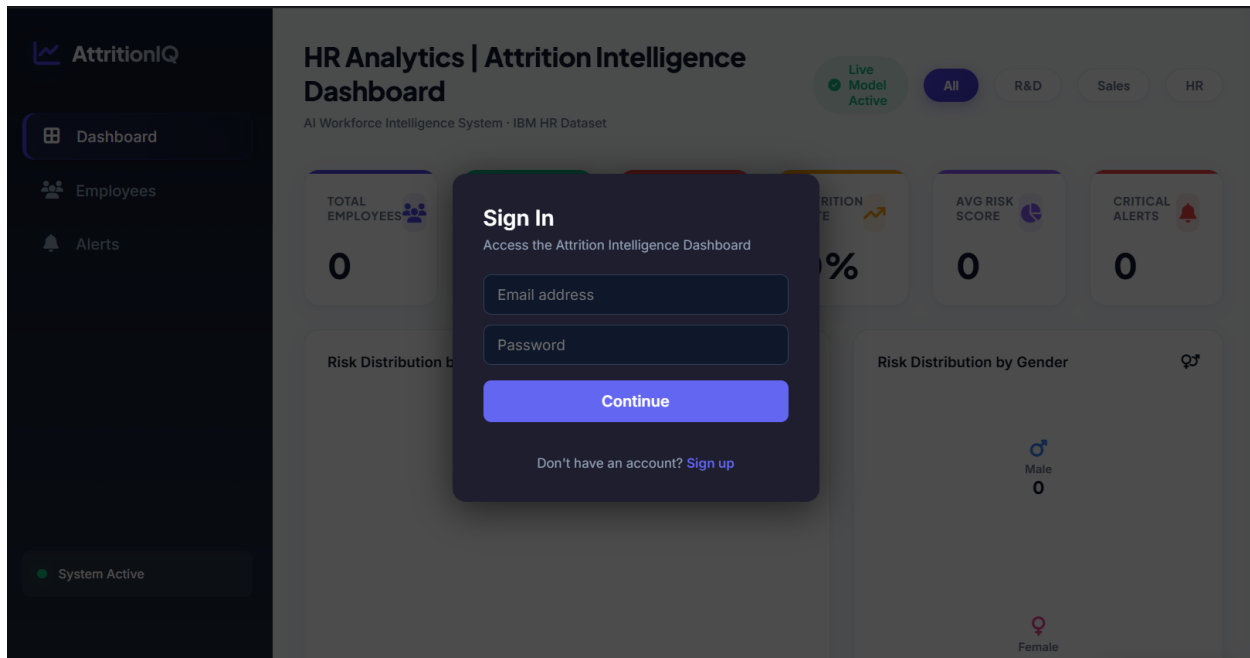
Important Notes:

- The Random Forest model (rf_model.joblib) is trained automatically on first startup if the model file does not exist. Subsequent startups load the pre-trained model for faster initialization.
- AI suggestions are cached in suggestions_cache.json. On repeat analysis runs with unchanged composite scores, cached suggestions are served instantly without calling the Groq API.
- The alert system (email and Slack) is disabled by default. Set EMAIL_ENABLED=True and SLACK_ENABLED=True in .env with valid SMTP and webhook credentials to activate alerts.

Exploring the Dashboard Web Pages:

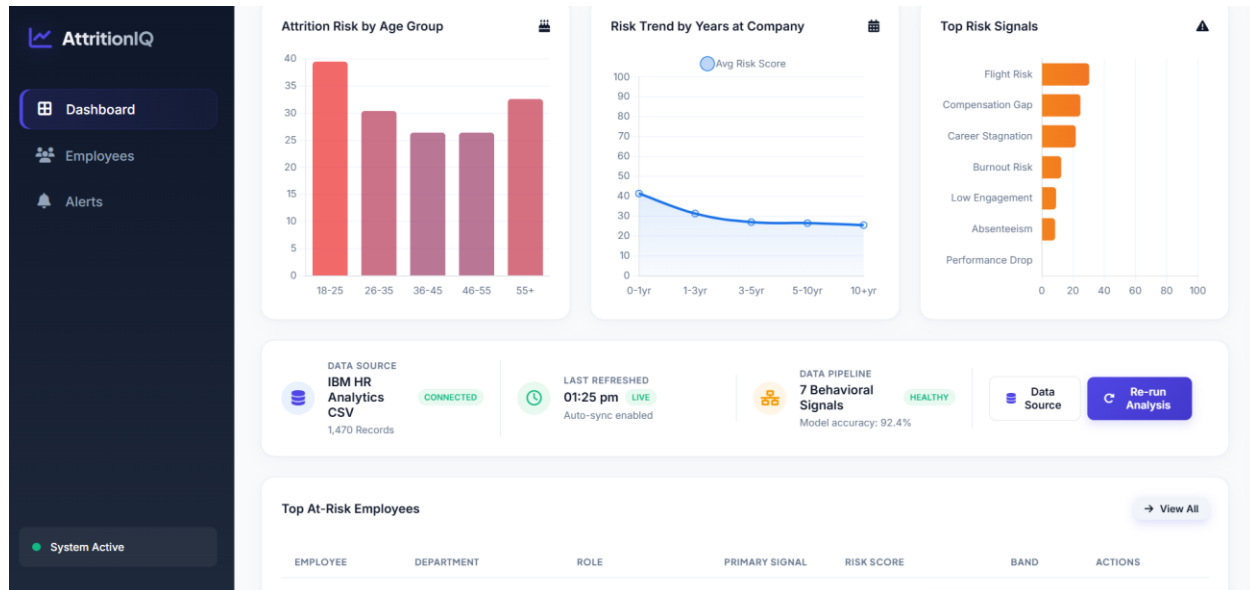
Overview / Dashboard Tab:

The main landing page of AttritionIQ serves as both the home view and the primary workforce intelligence command center. It features a dark indigo/slate sidebar navigation, a live model status badge, department filter buttons (All / R&D / Sales / HR), and a data pipeline health indicator. The main section displays 6 KPI cards, a stacked risk distribution bar chart by department, a gender distribution doughnut chart, and a top-5 at-risk employees table.



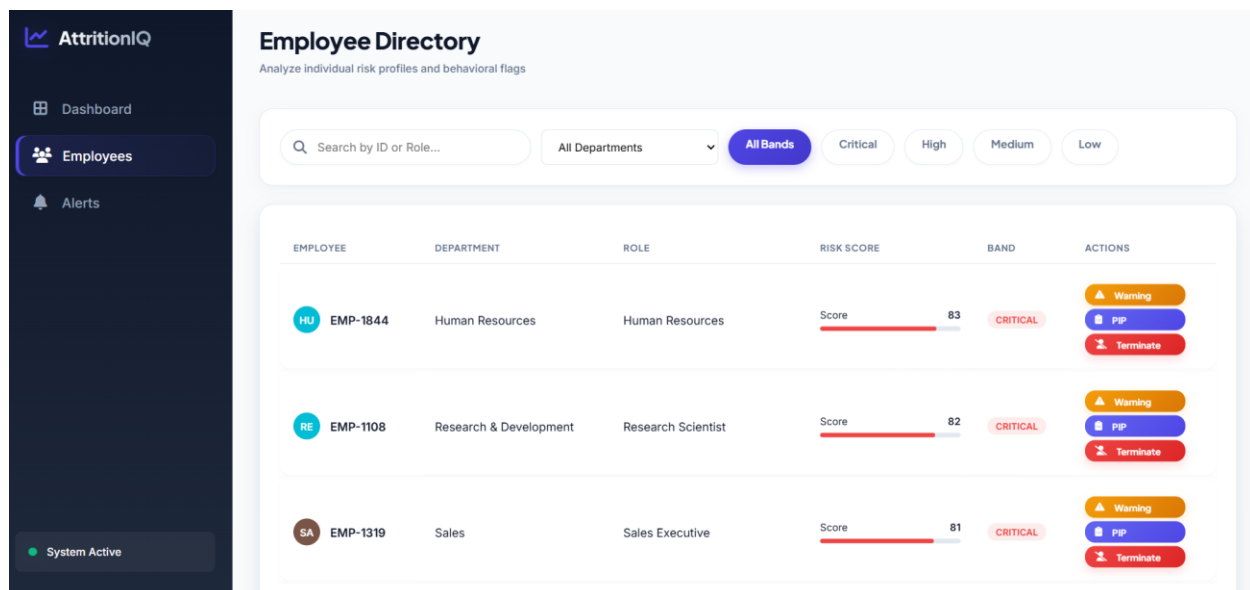
Overview Tab — KPI Cards and Risk Distribution Charts

[Screenshot: Dashboard showing 6 KPI cards (Total Employees: 1470, Active Employees: 1392, Total Risk: 78, Attrition Rate: 5.3%, Avg Risk Score: 29, Critical Alerts: 25), Risk Distribution by Department stacked bar chart, and Risk Distribution by Gender doughnut charts for Male (882) and Female (588).]



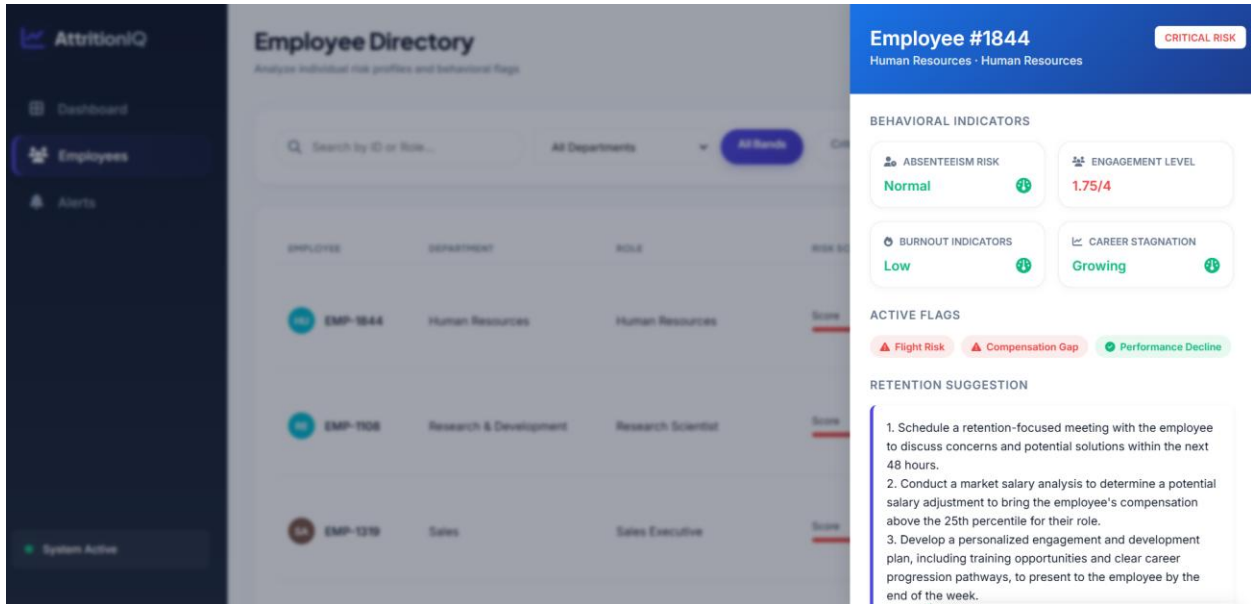
Employees Tab:

A searchable, filterable employee directory with risk band filter buttons (All Bands / Critical / High / Medium / Low) and department dropdown. Each employee row shows the employee ID, department, job role, risk score with a progress bar, risk band badge, and three inline action buttons: Warning Letter, PIP, and Terminate. Clicking any row opens the full employee detail side-panel.



Employee Detail Side-Panel:

A sliding side-panel triggered on employee row click, displaying the full employee profile: department, job role, behavioral indicators (Absenteeism Risk, Engagement Level, Burnout Indicators, Career Stagnation), all active behavioral flags (Flight Risk, Compensation Gap, Performance Decline), and the AI-generated retention suggestion from Groq LLaMA 3. The panel also includes a Schedule 1:1 Check-In action button.



The screenshot displays the AttritionIQ Employee Directory interface. The main panel shows a table of employees with columns for Employee ID, Department, Role, and Risk Score. A side-panel is open for Employee #1844, showing detailed information and behavioral indicators.

Employee #1844
Human Resources · Human Resources

BEHAVIORAL INDICATORS

- ABSENTEEISM RISK**: Normal
- ENGAGEMENT LEVEL**: 1.75/4
- BURNOUT INDICATORS**: Low
- CAREER STAGNATION**: Growing

ACTIVE FLAGS

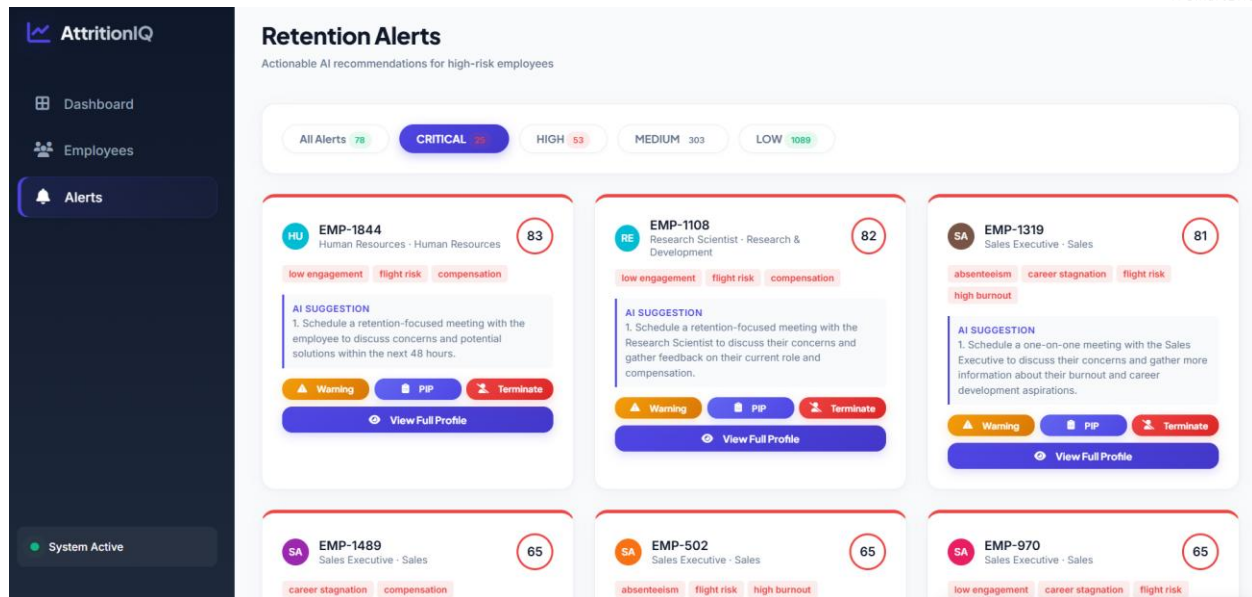
- Flight Risk
- Compensation Gap
- Performance Decline

RETENTION SUGGESTION

- Schedule a retention-focused meeting with the employee to discuss concerns and potential solutions within the next 48 hours.
- Conduct a market salary analysis to determine a potential salary adjustment to bring the employee's compensation above the 25th percentile for their role.
- Develop a personalized engagement and development plan, including training opportunities and clear career progression pathways, to present to the employee by the end of the week.

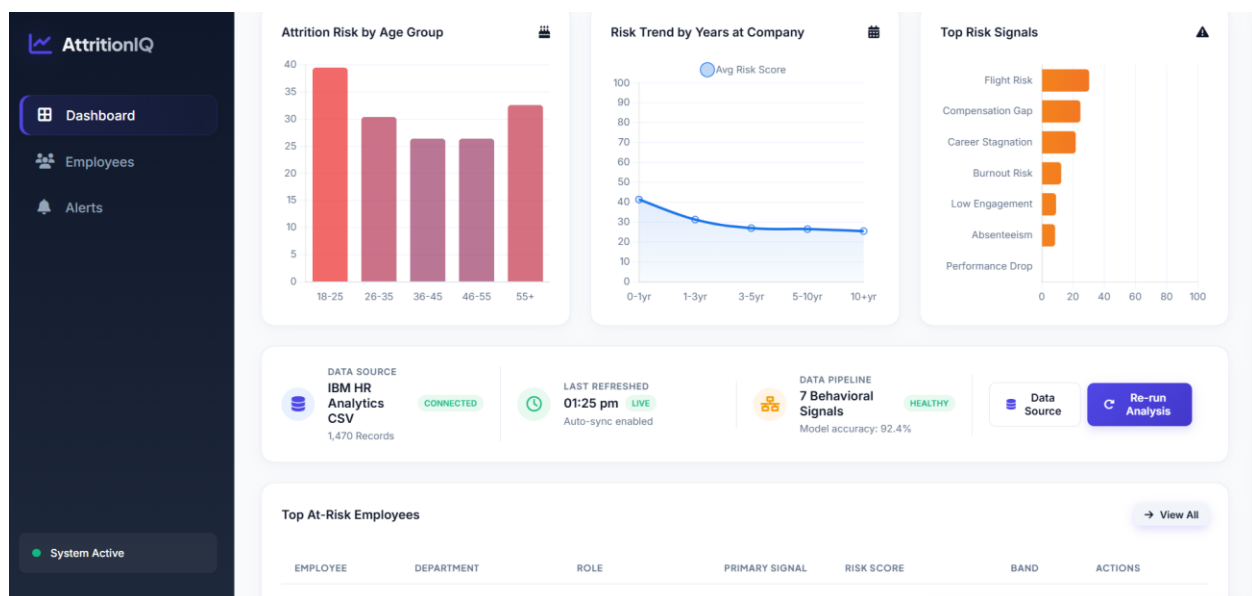
Alerts Tab:

A dedicated retention alerts page for all employees above the alert threshold (score ≥ 65). Employees are categorized into Critical, High, Medium, and Low tabs with counts. Each alert card shows the employee ID, department and role, active risk signal tags, composite score badge, and an AI-generated retention suggestion. Inline action buttons — Warning Letter, PIP, and Terminate — allow one-click HR document generation directly from the alert card. Generated documents appear in a modal and are downloadable as .txt files.



Analytics Tab:

An analytics view providing deeper workforce intelligence. Includes an Attrition Risk by Age Group bar chart (showing risk distribution across 18-25, 26-35, 36-45, 46-55, 55+ cohorts), a Risk Trend by Years at Company line chart (showing how composite risk evolves across 0-1yr, 1-3yr, 3-5yr, 5-10yr, 10+yr tenure bands), and a Top Risk Signals horizontal bar chart showing the prevalence of all 7 behavioral flags across the workforce. A Data Source panel at the bottom shows connection status, last refresh time, pipeline health, and Re-run Analysis and Data Source upload buttons.



Conclusion:

AttritionIQ is an AI-powered HR intelligence platform built with FastAPI and visualized through a polished single-file HTML/CSS/JS dashboard that streamlines proactive employee retention. It uses Groq's high-speed inference API with the LLaMA 3.3-70B Versatile model to instantly generate personalized, context-aware retention recommendations for high-risk employees, blended with a Random Forest Classifier and a rule-based behavioral scoring engine for composite risk assessment. The project — which included data engineering, ML pipeline development, LLM integration, dashboard visualization, and HR action document generation — highlights how targeted AI and a structured full-stack framework can transform reactive HR processes into proactive, data-driven retention intelligence. With planned enhancements including live HRMS integration, SHAP-based explainability, automated model retraining, and multi-tenant SaaS architecture, AttritionIQ is positioned to scale from a prototype to an enterprise-grade workforce intelligence product.